AI201 PA 3 Report
Implementing the Backpropagation Algorithm

Marcus Rafael B. Tiongson (2013-59474)

Date of Submission: March 27, 2026

1. INTRODUCTION

This report documents a from-scratch NumPy implementation of the backpropagation algorithm [1] applied to an eight-class classification problem. The dataset consists of 3 486 instances with 354 numerical features per instance and a heavily skewed label distribution: Class 1 accounts for 46.6% of samples, making straightforward gradient descent on the raw corpus prone to majority-class collapse. SMOTE [2, 3] is therefore applied prior to training to enforce uniform class frequency.

Two MLP configurations are evaluated under identical mini-batch training conditions, differing only in their hidden-layer activation functions: Hyperbolic Tangent (Network A) and Leaky ReLU (Network B). Both are paired with a Logistic output layer and trained with the generalised delta rule with momentum. The core experimental question is how sensitive convergence and final classification performance are to the choice of learning rate η, momentum coefficient α, and hidden-layer width H, given a fixed two-hidden-layer architecture and a 100-epoch budget.

2. OBJECTIVES

The primary objectives of this report are:

1. To implement the backpropagation learning algorithm for a fully-connected MLP with two hidden layers and one output layer, using only the NumPy, Matplotlib, OS, and CSV libraries.
2. To address severe class imbalance in the training set via SMOTE oversampling before model training.
3. To compare two activation-function regimes (Hyperbolic Tangent and Leaky Rectified Linear Unit) across their respective hidden layers, using the Logistic function at the output layer for all configurations.
4. To tune the key hyperparameters (learning rate η, momentum coefficient α, and hidden-layer width) so that the networks converge to a low mean square error and a high macro-averaged F1 score on the validation set.
5. To generate and export class predictions for the unlabelled test set using the best-performing trained model.

3. METHODOLOGY

3.1 Dataset Loading

The dataset is stored in three comma-separated files. `data.csv` contains 3 486 instances, each with 354 numerical features. `data_labels.csv` holds the corresponding integer class labels $y \in \{1, 2, \ldots, 8\}$, while `test_set.csv` contains the unlabelled test partition used for final prediction export. After loading, each label y_i is one-hot encoded into an $(N \times 8)$ matrix $\mathbf{D}$, where row i has a 1.0 at position $y_i - 1$ and 0.0 elsewhere:

$$D_{ik} = \begin{cases} 1.0 & \text{if } k = y_i - 1 \\ 0.0 & \text{otherwise} \end{cases} \tag{1}$$

3.2 Class Imbalance and SMOTE

The original dataset exhibits pronounced class imbalance, with Class 1 representing 46.6% of samples and Class 4 holding the smallest share. Training an MLP directly on such skewed data risks the network learning a trivial solution that always predicts the majority class, thereby achieving superficially high accuracy while entirely ignoring minority classes.

To mitigate this, SMOTE [2, 3] is applied. SMOTE generates synthetic minority-class samples by interpolating between a minority instance and one of its k-nearest neighbours in feature space. For each minority-class instance $\mathbf{x}_i$, a neighbour $\mathbf{x}_{nn}$ is chosen at random and a synthetic point $\mathbf{x}_{syn}$ is created as:

$$\mathbf{x}_{syn} = \mathbf{x}_i + \lambda\,(\mathbf{x}_{nn} - \mathbf{x}_i), \quad \lambda \sim \mathcal{U}(0, 1) \tag{2}$$

After applying SMOTE with `random_state=50`, every class holds an equal number of samples, yielding a balanced training corpus.

Note on data leakage: SMOTE was applied to the full labelled dataset prior to the train/validation split. Synthetic samples could therefore appear in both partitions, potentially leading to optimistic validation metrics. Ideally, SMOTE should be applied only within the training fold after partitioning; this trade-off was accepted for implementation simplicity.

Figure 1 shows the class distribution before and after SMOTE.

3.3 Dataset Partitioning

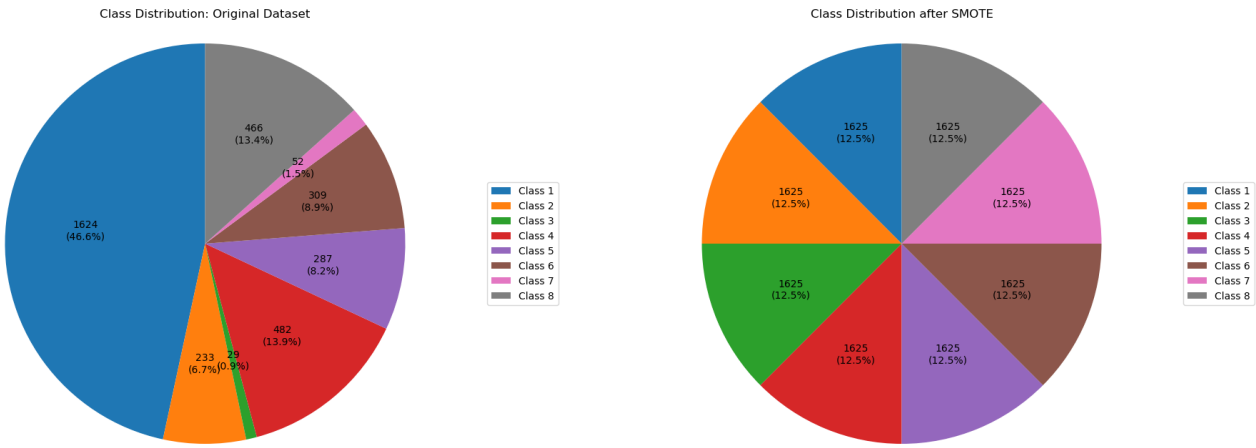

Figure 1: Class distribution of the original dataset (left) and after SMOTE oversampling (right). Class 1 accounts for 46.6% of the 3 486 original instances and Class 4 is the smallest minority. SMOTE raises all minority classes to 1 625 samples each, producing a perfectly uniform distribution of 13 000 instances total.

The balanced dataset is split into a **training set** and a **validation set**. Exactly 800 samples are drawn uniformly at random (without replacement, `random_state=50`) as the validation set; all remaining samples form the training set. Both partitions are exported to CSV for reproducibility.

3.4 Network Architecture

The MLP consists of three learnable layers:

- **Hidden layer i:** receives a bias-augmented input vector and maps it to a vector of size H.
- **Hidden layer j:** receives the bias-augmented output of layer i and maps it to a vector of size H.
- **Output layer k:** receives the bias-augmented output of layer j and produces 8 output activations, one per class.

Bias units are handled implicitly: a constant $\varphi_0 = +1$ is prepended to every layer's input vector, so the corresponding weight $w_{j0}^{(l)}$ acts as the bias for neuron j in layer l. Weight matrices are initialised from a zero-mean Gaussian with standard deviation 0.01.

3.5 Forward Propagation

For layer $l \in \{i, j, k\}$, the *internal activity* (net input) of neuron j is:

$$v_j^{(l)}(n) = \sum_{m=0}^{p} w_{jm}^{(l)}(n)\, \varphi_m\left(v_m^{(l-1)}(n)\right) \tag{3}$$

where p is the width of layer $l-1$, the index $m = 0$ corresponds to the bias unit ($\varphi_0 = +1$), and n denotes the current training step. The *neuron output* is then:

$$y_j^{(l)}(n) = \varphi_j\left(v_j^{(l)}(n)\right) \tag{4}$$

3.6 Activation Functions

Three activation functions are studied. Derivatives are expressed directly in terms of the layer output $o = \varphi(v)$ to avoid recomputing v during backpropagation.

3.6.1 *Logistic (Sigmoid)*

$$\varphi(v) = \frac{1}{1 + e^{-av}}, \quad a > 0 \tag{5}$$

$$\varphi'(v) = a \cdot o\,(1 - o) \tag{6}$$

The parameter $a = 2.0$ is used at the output layer for all network configurations. Larger a steepens the sigmoid.

3.6.2 *Hyperbolic Tangent (Tanh)*

$$\varphi(v) = a \tanh(bv) = a \cdot \frac{e^{bv} - e^{-bv}}{e^{bv} + e^{-bv}} \tag{7}$$

$$\varphi'(v) = \frac{b}{a}\,(a - o)\,(a + o) \tag{8}$$

Following standard practice, the constants $a = 1.716$ and $b = 0.66667$ are used for the hidden layers of Network A.

3.6.3 *Leaky ReLU*

$$\varphi(v) = \begin{cases} v & \text{if } v > 0 \\ \gamma v & \text{if } v \le 0 \end{cases}, \quad \gamma \in (0, 1) \tag{9}$$

$$\varphi'(v) = \begin{cases} 1 & \text{if } o > 0 \\ \gamma & \text{if } o \le 0 \end{cases} \tag{10}$$

With $\gamma = 0.01$, negative inputs receive a small but non-zero gradient, preventing the "dying ReLU" problem. Gradient clipping (norm threshold 5.0) is applied to Leaky ReLU hidden layers to guard against exploding gradients.

3.7 Error Computation

The *error signal* at output neuron k is the difference between the desired output d_k and the actual output o_k:

$$e_k(n) = d_k(n) - o_k(n) \tag{11}$$

The *mean square error* (MSE) over all C output neurons at step n is:

$$\mathcal{E}(n) = \frac{1}{2} \sum_{k=1}^{C} e_k^2(n) \tag{12}$$

3.8 Backpropagation Delta Equations

3.8.1 Output Layer Delta

The local gradient (delta) for output neuron k is:

$$\delta_k^{(L)}(n) = e_k(n) \cdot \varphi'\Big(v_k^{(L)}(n)\Big) \tag{13}$$

3.8.2 Hidden Layer Delta

For hidden layer $l \in \{L-1, L-2, \ldots, 1\}$:

$$\delta_j^{(l)}(n) = \varphi'\Big(v_j^{(l)}(n)\Big) \cdot \sum_k \delta_k^{(l+1)}(n)\, w_{kj}^{(l+1)}(n) \tag{14}$$

The summation propagates the error signal backward from layer $l+1$, weighted by the connections $w_{kj}^{(l+1)}$. Bias weights (column 0) are excluded from this sum because the bias neuron carries no incoming error.

3.9 Weight Update: Generalised Delta Rule with Momentum

The weight correction for connection $w_{ji}^{(l)}$ follows the *negative gradient* of $\mathcal{E}$ with respect to that weight:

$$-\frac{\partial \mathcal{E}(n)}{\partial w_{ji}^{(l)}(n)} = \delta_j^{(l)}(n) \cdot \varphi_i\Big(v_i^{(l-1)}(n)\Big) \tag{15}$$

The full update rule adds a momentum term that carries forward a fraction α of the previous weight increment:

$$\Delta w_{ji}^{(l)}(n) = \alpha\, \Delta w_{ji}^{(l)}(n-1) + \eta\, \delta_j^{(l)}(n)\, \varphi_i\Big(v_i^{(l-1)}(n)\Big) \tag{16}$$

$$w_{ji}^{(l)}(n+1) = w_{ji}^{(l)}(n) + \Delta w_{ji}^{(l)}(n) \tag{17}$$

Bias weights ($i = 0$, with $\varphi_0 = +1$) are updated identically:

$$\Delta w_{j0}^{(l)}(n) = \alpha\, \Delta w_{j0}^{(l)}(n-1) + \eta\, \delta_j^{(l)}(n) \tag{18}$$

When $\alpha = 0$ the rule reduces to vanilla gradient descent. When $\alpha > 0$ it smooths oscillations in the weight space and can accelerate convergence along consistent gradient directions.

3.10 Mini-Batch Gradient Descent

Rather than updating weights after every sample (online) or after the full dataset (batch), a *mini-batch* strategy with batch size $B = 8$ is used. Gradients are accumulated across all samples in a batch and then the average gradient is applied:

$$\Delta w_{ji}^{(l)}(n) = \alpha\, \Delta w_{ji}^{(l)}(n-1) + \frac{\eta}{B} \sum_{s=1}^{B} \delta_{j,s}^{(l)}(n)\, \varphi_{i,s}\Big(v_i^{(l-1)}(n)\Big) \tag{19}$$

Mini-batch training reduces gradient variance compared to online updates and is more computationally efficient than full-batch gradient descent.

3.11 Validation and Evaluation Metrics

After each epoch the model is evaluated on the 800-sample validation set. Predictions are determined by the arg max of the output layer activations. The following metrics are computed from the resulting 8×8 confusion matrix:

- **Accuracy** $= \dfrac{\mathrm{TP}_{\mathrm{total}}}{N}$
- **Macro-averaged Precision, Recall, F1** computed per class and then averaged uniformly across all eight classes.
- **Matthews Correlation Coefficient (MCC)** for multiclass classification, defined as:

$$\mathrm{MCC} = \frac{c\, s - \sum_k p_k t_k}{\sqrt{\left(s^2 - \sum_k t_k^2\right)\left(s^2 - \sum_k p_k^2\right)}} \tag{20}$$

  where s is the total number of samples, c is the number correctly classified, t_k is the actual count of class k, and p_k is the predicted count of class k.

4. EXPERIMENTAL RESULTS

4.1 Hyperparameter Configurations

Two base network architectures are evaluated: **Network A** uses Tanh activation ($a = 1.716$, $b = 0.66667$) in both hidden layers, and **Network B** uses Leaky ReLU ($\gamma = 0.01$) in both hidden layers. Both networks use a Logistic output layer ($a = 1.0$). Each family is evaluated under five settings: the default configuration, a first stable run at $\eta = 0.10$ and $\alpha = 0.50$, a second run with a lower learning rate $\eta = 0.05$, a high-momentum "fast" variant with $\alpha = 0.90$, and a reduced-width $H = 5$ variant.

All configurations are trained for 100 epochs using mini-batch size $B = 8$. After SMOTE and the fixed 800-sample validation split, the training set contains 12 200 samples, so each run performs 1 525 mini-batch updates per epoch, or 152 500 updates over 100 epochs.

Table 1: Hyperparameter Configurations Tested

Configuration	η	α	H	Activation
`networkA`	0.85	0.90	8	Tanh
`networkA_improv`	0.10	0.50	12	Tanh
`networkA_improv2`	0.05	0.50	12	Tanh
`networkA_improv2_fast`	0.05	0.90	12	Tanh
`networkA_improv2_small`	0.05	0.90	5	Tanh
`networkB`	0.85	0.90	8	Leaky ReLU
`networkB_improv`	0.10	0.50	12	Leaky ReLU
`networkB_improv2`	0.05	0.50	12	Leaky ReLU
`networkB_improv2_fast`	0.05	0.90	12	Leaky ReLU
`networkB_improv2_small`	0.05	0.90	5	Leaky ReLU

4.2 Training and Validation Errors

The two default runs never entered a useful learning regime. For `networkA`, the training/validation MSE moved from (0.808, 0.867) at epoch 1 to a plateau near (0.875, 0.867) and never improved thereafter. `networkB` behaved slightly differently: it briefly reached minima of approximately (0.401, 0.394), but then drifted upward and finished near (0.874, 0.889). In both cases, the models failed to satisfy the notebook's convergence criterion that both training and validation MSE drop below 0.02 within the 100-epoch budget.

Lower learning rates of 0.10 and 0.05 were what made learning possible, but the two activation families still behaved differently. The tuned Tanh models converged more smoothly, and all four non-default variants eventually met the 0.02 criterion. The earliest was `networkA_improv2_fast`, which crossed the threshold at epoch 44. The tuned Leaky ReLU models achieved the highest final classification scores, but their validation curves were visibly more erratic. The best overall model, `networkB_improv2`, reached the threshold only at epoch 73, while `networkB_improv2_fast` reduced this to epoch 51 at a small cost in F1 score. The reduced-width Leaky ReLU model, `networkB_improv2_small`, remained unstable and never converged.

4.3 Scores Summary

Table 2 reports the final evaluation metrics on the 800-sample validation set for all ten configurations, ranked by the same macro-F1 criterion used in the notebook's *Training Results* section.

The ranking separates the tuned models from the failed ones clearly. Seven configurations achieved macro F1 above 0.96, and three exceeded 0.98, whereas the two defaults and the reduced-width Leaky ReLU run fell far outside that cluster. If convergence speed is also considered, `networkA_improv2_fast` offers the strongest efficiency–accuracy compromise: it reaches the target error in 44 epochs and still attains macro F1 = 0.9835. Although `networkA_improv` has a slightly smaller time-adjusted cost (29.26 s versus 29.70 s), the higher-momentum Tanh model gives a clearer gain in predictive quality for nearly the same normalized training expense.

4.4 Best Model and Prediction

`networkB_improv2` is the best overall model. It uses Leaky ReLU with $\eta = 0.05$, $\alpha = 0.50$, and $H = 12$, and achieves 98.875% accuracy, MCC = 0.98714, and macro F1 = 0.98861 on the validation set. Within the Tanh family, the selected model is `networkA_improv2_fast`, which uses $\eta = 0.05$, $\alpha = 0.90$, and $H = 12$; it converges much earlier at epoch 44 while still reaching macro F1 = 0.98352.

These are also the two models exported, ready for production. The Tanh weights are saved to `modelA/trained_weights.csv`, the Leaky ReLU weights to `modelB/trained_weights.csv`, and their respective 701-sample test-set predictions are written to `predictions/predictions_for_test_tanh.csv` and `predictions/predictions_for_test_leakyrelu.csv`. The distinction is important: the Leaky ReLU model is the best by absolute validation score, whereas the Tanh model is retained because it gives the strongest speed–performance compromise within its activation family.

5. ANALYSIS AND DISCUSSION OF RESULTS

5.1 Impact of Learning Rate

The single most consequential hyperparameter change was reducing the learning rate from $\eta = 0.85$ to $\eta = 0.10$. At 0.85, the weight increments $\Delta w_{ji}^{(l)}$ (Equation 16) were so large that the optimiser repeatedly jumped over the minimum of $\mathcal{E}$, producing plateaued or unstable error curves and near-zero MCC scores. Both the Tanh and Leaky ReLU networks were equally vulnerable to this instability, underscoring that activation function choice is secondary to choosing an appropriate step size.

5.2 Role of Momentum

The notebook results show that momentum only became useful after the learning rate had already been reduced to a stable range. Holding $\eta = 0.05$ and $H = 12$ fixed, increasing α from 0.50 to 0.90 reduced the Tanh convergence epoch from 74 in `networkA_improv2` to 44 in `networkA_improv2_fast`, while also improving macro F1 from 0.97609 to 0.98352. For Leaky ReLU, the same change reduced the convergence epoch from 73 in `networkB_improv2` to 51 in `networkB_improv2_fast`, but at a slight cost in macro F1 (0.98861 down to 0.98530).

Momentum acts like an "epoch-skipping" term: it compresses the loss curve horizontally and is best viewed as a convergence-speed control rather than as a guarantee of a better final optimum. High momentum was not enough to save the default $\eta = 0.85$ runs, so it should be tuned only after the step size is already well behaved.

Table 2: Validation-set summary ranked by macro F1. "Epoch" denotes the first epoch at which both training and validation MSE fell below 0.02; DNF indicates that this did not occur within 100 epochs. Time-adjusted cost is computed as Time × (Epoch/100) for converged runs.

Configuration	Epoch	F1	MCC	Acc.	Time-adj. (s)
`networkB_improv2`	73	0.98861	0.98714	0.98875	70.75
`networkB_improv2_fast`	51	0.98530	0.98286	0.98500	44.58
`networkA_improv2_fast`	44	0.98352	0.98011	0.98250	29.70
`networkA_improv2_small`	74	0.98108	0.97860	0.98125	31.34
`networkA_improv`	56	0.98006	0.97725	0.98000	29.26
`networkA_improv2`	74	0.97609	0.97309	0.97625	43.91
`networkB_improv`	71	0.96129	0.95481	0.96000	52.80
`networkB_improv2_small`	DNF	0.55645	0.59344	0.61250	N/A
`networkA`	DNF	0.02925	0.00000	0.13250	N/A
`networkB`	DNF	0.02503	0.00000	0.11125	N/A

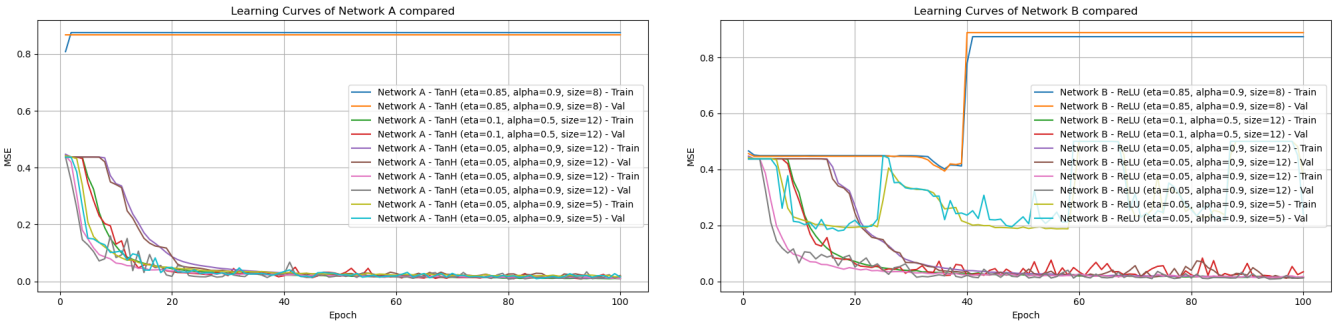

Figure 2: Training and validation MSE for the five Tanh configurations (left) and five Leaky ReLU configurations (right). Lowering the learning rate enables convergence in both families, higher momentum compresses the number of epochs needed once the learning rate is stabilized, and the 5-node Leaky ReLU model is the clearest failure case.

5.3 Hidden Layer Width

The width experiments in the notebook compare $H = 12$ against a reduced $H = 5$ model, not against an intermediate width. At $\eta = 0.05$ and $\alpha = 0.90$, shrinking the Tanh network from 12 to 5 hidden units reduced raw training time from 67.50 s to 42.36 s, but also delayed convergence from epoch 44 to epoch 74 and lowered macro F1 from 0.98352 to 0.98108. The smaller Tanh model therefore remains viable only when compute is constrained.

The same reduction was far more damaging for Leaky ReLU. `networkB_improv2_small` never achieved the 0.02 MSE target and collapsed to macro F1 = 0.55645, far below the 0.98861 attained by the 12-node `networkB_improv2`. Hidden-layer width therefore interacts strongly with activation choice: $H = 12$ is the safer default, while $H = 5$ is defensible only for the Tanh family and only as a hardware-driven compromise.

5.4 Activation Function Comparison

Leaky ReLU produced the strongest peak validation scores, taking the top two positions in Table 2 through `networkB_improv2` and `networkB_improv2_fast`. The best Tanh model, `networkA_improv2_fast`, ranked third overall with macro F1 = 0.98352, roughly 0.51 percentage points below the best Leaky ReLU result. If the comparison stopped at peak F1, Leaky ReLU would be the clear winner.

The learning-curve evidence adds a more nuanced interpretation. The notebook commentary correctly notes that Leaky ReLU exhibited more erratic validation behavior than Tanh, and the small-width ReLU variant failed completely. Tanh converged more smoothly, tolerated the reduced $H = 5$ setting, and delivered a much better efficiency–accuracy trade-off in `networkA_improv2_fast`. The practical conclusion is therefore twofold: Leaky ReLU is the better choice when absolute predictive performance is the priority, while Tanh is the more robust choice when convergence smoothness and computational efficiency also matter.

5.5 Effect of SMOTE on Class Balance

Applying SMOTE prior to training yielded a perfectly uniform class distribution, which directly enabled the network to learn discriminative features for all eight classes. Without class balancing, Class 1 (46.6% of data) would dominate the gradient signal, causing the network to converge to a trivial majority-class predictor, consistent with the near-zero MCC observed under default hyperparameters before balancing. However, the leakage introduced by balancing before splitting means the 98.6% validation accuracy is likely slightly optimistic compared to what would be observed on truly unseen data.

5.6 Convergence Rate and Early Stopping

The fast variants captured most of their usable gains early: `networkA_improv2_fast` met the target at epoch 44 and `networkB_improv2_fast` at epoch 51. The highest-F1 mod-

els, however, required longer runs: `networkB_improv2` did not cross the threshold until epoch 73, and several Tanh variants also converged only after epoch 56. Early stopping is therefore still sensible, but its patience should be tuned per configuration rather than fixed at a single aggressive cutoff such as epoch 40.

6. CONCLUSION

This work demonstrated a complete, from-scratch implementation of the backpropagation algorithm on an eight-class imbalanced dataset. The following conclusions are drawn:

1. **The default configuration was not usable.** The baseline `networkA` and `networkB` runs both failed to converge and finished with macro F1 scores of only 0.02925 and 0.02503, respectively. The original $\eta = 0.85$ and $\alpha = 0.90$ setting was simply too aggressive for this task.
2. **Lowering the learning rate was the main factor that made training succeed.** Once η was reduced to 0.10 or 0.05, both activation families produced strong models, and the best overall result came from `networkB_improv2` with macro F1 = 0.98861.
3. **Higher momentum improved convergence speed only after the learning rate had been stabilized.** Raising α from 0.50 to 0.90 accelerated both activation families, but its effect was primarily on epochs-to-convergence rather than on the final score.
4. **A 12-node hidden layer was more reliable than a 5-node hidden layer.** The smaller Tanh network remained acceptable, but the smaller Leaky ReLU network failed badly. Hidden-layer width should therefore be treated as a capacity constraint, not just a speed knob.
5. **Leaky ReLU delivered the best peak accuracy, while Tanh provided the smoother efficiency–accuracy tradeoff.** When prediction quality is the sole criterion, `networkB_improv2` is the best model; when faster convergence also matters, `networkA_improv2_fast` is the strongest Tanh choice.
6. **SMOTE effectively neutralized class imbalance,** enabling all eight classes to contribute proportionally to the gradient signal. Future work should apply SMOTE only to the training fold, after the train/validation split, to remove leakage from the validation set.

Based on these results, the following recommendations are proposed:

1. Use `networkB_improv2` as the primary model when the objective is maximum validation performance, since it achieved the highest macro F1 score among all tested configurations.
2. Prefer a reduced learning rate in the range $\eta \in \{0.10, 0.05\}$ for future tuning, because the experiments showed that stable convergence depended far more on this choice than on any other hyperparameter.
3. Retain higher momentum only after a stable learning rate has been selected. In this study, $\alpha = 0.90$ was useful for faster convergence, but it was not sufficient to rescue an unstable training setup on its own.
4. Avoid undersized hidden layers when working with this dataset. A width of 12 hidden units was consistently more dependable than 5 units, especially for the Leaky ReLU network.
5. Revise the experimental pipeline so that SMOTE is applied only after the train/validation split. This would produce a more reliable estimate of generalization performance and remove leakage from model evaluation.
6. Evaluate the implementation on computing environments with fewer hardware limitations than personal laptops, such as laboratory workstations or shared high-performance systems. This would allow broader hyperparameter searches, longer runs, and more consistent timing measurements without being constrained by local memory and processing limits.
7. Conduct an epoch-driven follow-up study that explicitly compares how quickly different configurations converge across a wider range of setups. Instead of reporting only the final validation score, future experiments should record the epoch at which each model reaches predefined performance thresholds, making it possible to rank networks not only by accuracy but also by convergence speed.
8. Extend the study with multiple random splits or cross-validation to confirm whether the observed advantage of Leaky ReLU over Tanh remains consistent beyond a single partition of the data.

7. REFERENCES

[1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986.

[2] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[3] J. Brownlee, "SMOTE for imbalanced classification with Python," *Machine Learning Mastery*, 2021. [Online]. Available: `https://machinelearningmastery.com/smote-oversampling-for-imbalanced-classification/`

APPENDIX

A. CLASS-DISTRIBUTION PIE CHARTS

These supplementary pie charts show the post-SMOTE class composition of the training and validation splits.

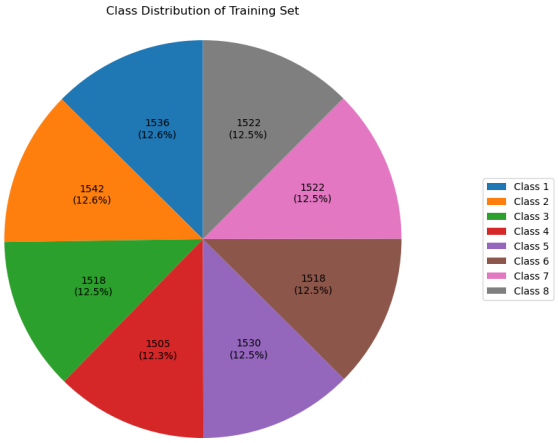

Figure 3: Training-split class distribution after SMOTE.

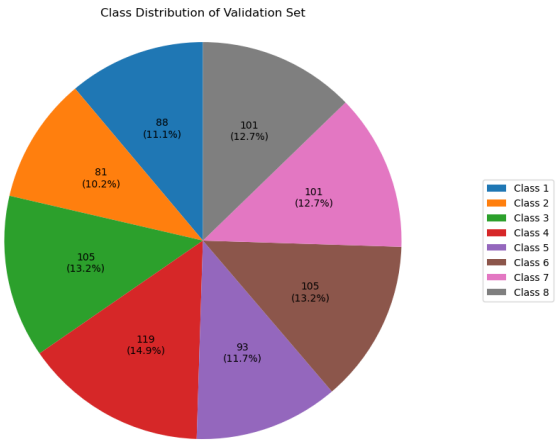

Figure 4: Validation-split class distribution after SMOTE.

B. LEARNING CURVES: NETWORK A

These figures collect the individual learning curves that support the family-level comparison in Figure 2 for Network A.

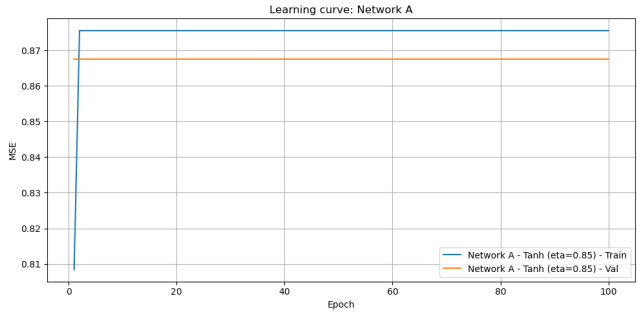

Figure 5: Learning curve for `networkA`. The default Tanh model does not converge.

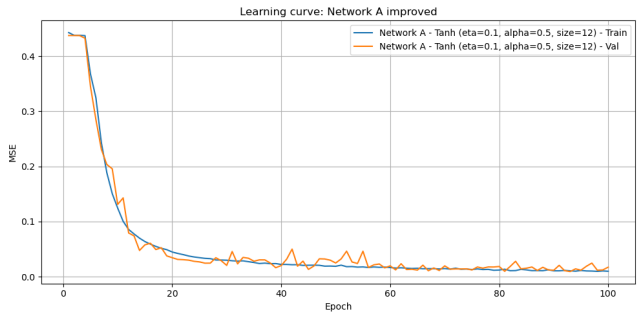

Figure 6: Learning curve for `networkA_improv`. Lowering the learning rate restores stable Tanh training.

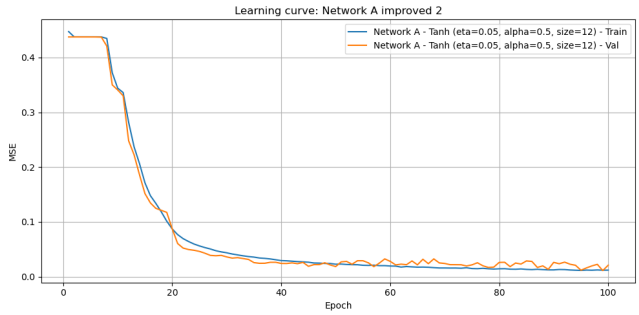

Figure 7: Learning curve for `networkA_improv2`. The lower learning rate keeps the loss trajectory smooth.

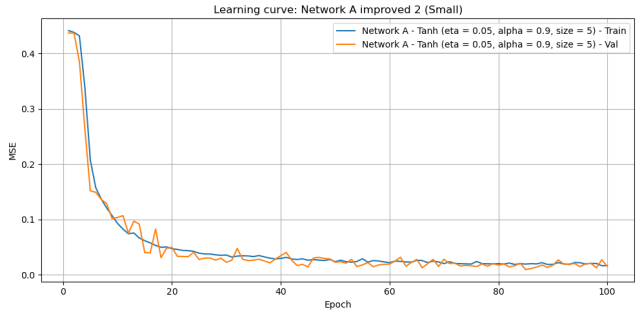

Figure 8: Learning curve for `networkA_improv2_fast`. Higher momentum produces earlier convergence.

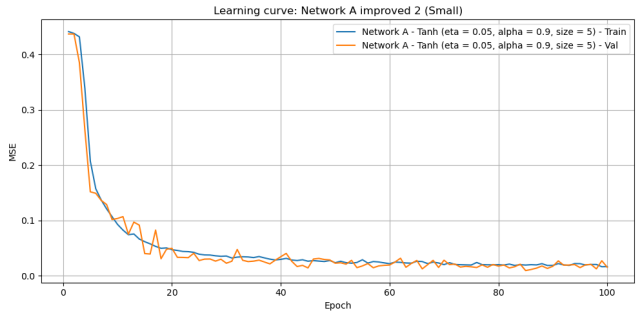

Figure 9: Learning curve for `networkA_improv2_small`. The $H = 5$ Tanh model still converges, but more slowly.

C. LEARNING CURVES: NETWORK B

These figures collect the individual learning curves that support the family-level comparison in Figure 2 for Network B.

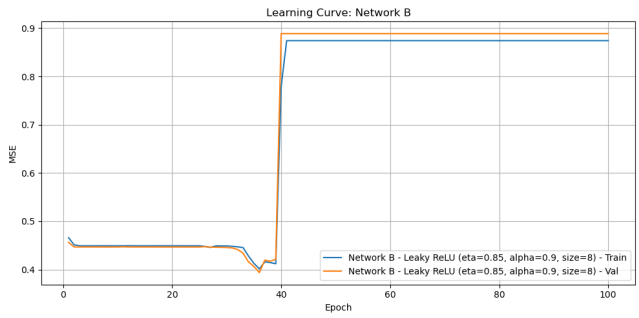

Figure 10: Learning curve for `networkB`. The default Leaky ReLU model remains unstable.

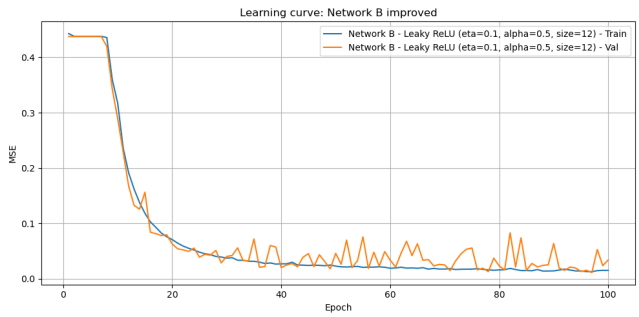

Figure 11: Learning curve for `networkB_improv`. The tuned model learns effectively after the learning-rate reduction.

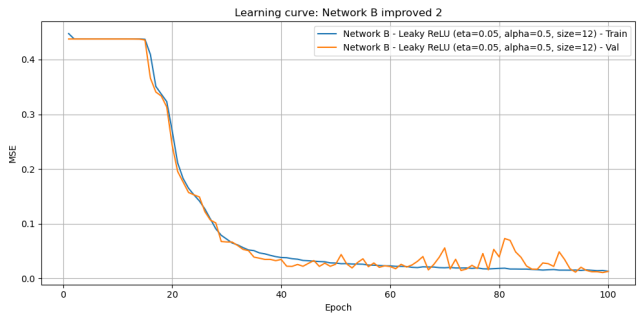

Figure 12: Learning curve for `networkB_improv2`. This run gives the best final F1.

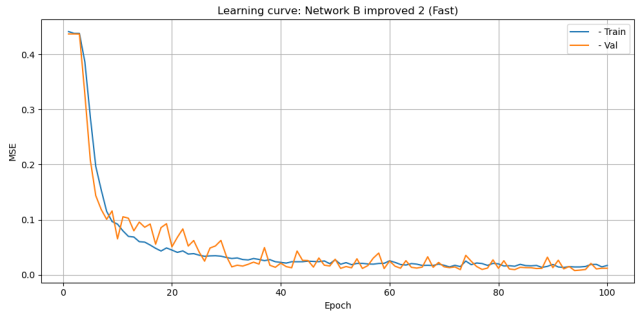

Figure 13: Learning curve for `networkB_improv2_fast`. The fast variant converges earlier with a small score tradeoff.

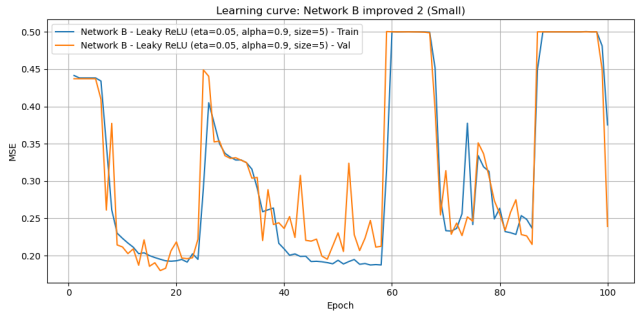

Figure 14: Learning curve for `networkB_improv2_small`. The $H = 5$ configuration is the clearest tuned failure case.